

Programmation et Algorithmique I

Test du 3 novembre 2010

Veillez respecter scrupuleusement les consignes suivantes :

- répondez à chaque problème **sur des feuilles séparées** et utilisez uniquement les feuilles de réponses qui vous sont données ;
- indiquez votre nom, prénom et section sur chaque feuille de réponses ;
- les notes de cours et personnelles **ne sont pas autorisées** ;
- si ce n'est pas explicitement demandé dans l'énoncé, **vous ne devez pas tester que les préconditions sont respectées**.

Problème 1 : complexité

Soit une fonction `f` qui prend une liste `t` en entrée et qui retourne un nombre entier (la numérotation des lignes ne fait pas partie du code) :

```
1 def f(t):
2     if est_acceptee(len(t)):
3         return reduit(t, 1)
4     else:
5         sum = 0
6         for item in t:
7             sum += reduit(t, item)
8         return sum
```

Cette fonction utilise les fonctions `est_accepte` et `reduit`. On ne connaît pas le travail réel de ces fonctions (il n'a pas d'importance pour cette question) mais l'on sait que :

- la fonction booléenne `est_accepte(k)` retourne `True` ou `False` en fonction de la valeur de l'entier `k` passé en paramètre.
- la fonction `reduit(liste, p)` prend une liste `liste` et un entier `p` en entrée et effectue une réduction sur cette liste : elle retourne un entier.

Questions

On vous demande de calculer la complexité dans le pire des cas de `f(t)` en fonction du nombre n d'éléments dans la liste `t` pour chacune des hypothèses suivantes. Vos réponses doivent être justifiées.

- a) on suppose que les fonctions `est_accepte(k)` et `reduit(t, p)` ont toutes 2 une complexité en $\mathcal{O}(1)$.
- b) on suppose que `est_accepte(k)` est en $\mathcal{O}(1)$ et que `reduit(liste, p)` est en $\mathcal{O}(m)$ où m est le nombre d'éléments dans la liste `liste`.
- c) on suppose que `est_accepte(k)` est en $\mathcal{O}(k^2)$ et que `reduit(liste, p)` est en $\mathcal{O}(\sqrt{m})$ où m est le nombre d'éléments dans la liste `liste`.

Problème 2 : exactitude

On considère la fonction suivante, qui a pour but de calculer la somme des éléments stockés dans une liste d'entiers `liste` :

```
def sum(liste):
    n = len(liste)
    somme = 0
    i = 0
    while i < n:
        somme = somme + liste[i]
        i = i + 1
    return somme
```

Questions

- a) Prouvez que la fonction `sum` s'arrête toujours.
- b) Parmi les assertions suivantes, donnez les deux qui **ne constituent pas** un invariant de la boucle ci-dessus. Justifiez votre réponse. Prouvez ensuite que l'assertion restante est un invariant de cette boucle.
 - (a) $i \leq n$ et $somme = \sum_{j=0}^{i-1} liste[j]$
 - (b) $i \leq n - 1$ et $somme = \sum_{j=0}^{i-1} liste[j]$
 - (c) $somme = \sum_{j=0}^i liste[j]$
- c) Prouvez que la fonction `sum` est correcte, c'est-à-dire qu'elle calcule effectivement la somme des éléments de la liste.

Problème 3 : Nombres premiers

Un *nombre premier* est un entier strictement positif qui admet exactement deux diviseurs **distincts** entiers et positifs. Ces diviseurs sont 1 et lui-même.

Pour vérifier qu'un nombre entier strictement plus grand que 1 est un nombre premier, il suffit de tester qu'il n'est divisible par aucun des nombres premiers inférieurs ou égaux à sa racine carrée.

Questions

Donnez le code Python des fonctions suivantes :

- a) Une fonction `est_premier(candidat, premiers)` qui prend en paramètre un entier strictement positif `candidat`, et une liste `premiers`. Cette liste `premiers` contient uniquement des nombres premiers, triés par ordre croissant. De plus, cette liste contient au moins tous les nombres premiers inférieurs strictement à `candidat`.

La fonction doit retourner `True` si `candidat` est un nombre premier, `False` sinon. Pour y parvenir veuillez implémenter la technique explicitée plus tôt.

- b) Une fonction `generer_nombres_premiers(nb)` qui prend un nombre entier positif `nb` en paramètre. Cette fonction doit retourner la liste des `nb` premiers *nombres premiers*. Pour y parvenir, veuillez vous servir de la fonction `est_premier`. Faites attention de respecter les préconditions de cette fonction.

Problème 4 : Opérations matricielles

Nous représentons une matrice $(n \times m)$ par une liste (de longueur n) de listes (de longueur m). Nous obtenons alors la représentation suivante :

$$\begin{pmatrix} a_{0,0} & \dots & a_{0,m-1} \\ \vdots & \ddots & \vdots \\ a_{n-1,0} & \dots & a_{n-1,m-1} \end{pmatrix} \Leftrightarrow [[a_{0,0}, \dots, a_{0,m-1}], [a_{1,0}, \dots, a_{1,m-1}], \dots, [a_{n-1,0}, \dots, a_{n-1,m-1}]]$$

Pour accéder à l'élément $a_{i,j}$ ($0 \leq i < n$ et $0 \leq j < m$) d'une matrice `A`, nous écrivons donc : `A[i][j]`.

Questions

Donnez le code Python des fonctions suivantes :

- a) Une fonction `add(m1, m2)` qui retourne une nouvelle matrice correspondant à la somme des matrices `m1` et `m2`. Si les tailles des matrices ne sont pas compatibles, retourne `None`.
- b) Une fonction `multiply(m1, m2)` qui retourne une nouvelle matrice correspondant au produit des matrices `m1` et `m2`. Si les tailles des matrices ne sont pas compatibles, retourne `None`.

Notez que ces fonctions **ne peuvent pas présenter des effets de bord**.

Problème 5 : Chaînes de caractères et récursivité

Les questions ci-dessous portent sur les chaînes de caractères. Il vous est demandé d'écrire certaines fonctions sur les chaînes de caractères de manière récursive.

Questions

Donnez le code Python des fonctions suivantes :

- a) Une fonction `récursive contient(mot, caractere)` qui renvoie `True` si la chaîne `mot` contient le caractère `caractere`, et `False` sinon. Vous ne pouvez pas utiliser l'opérateur `in`, les fonctions `count`, `find`, `index`, les boucles `for` et `while` ni leurs variantes.

- b) Une fonction **récursive** `palindrome(mot)` qui retourne `True` si la chaîne `mot` est un palindrome, et `False` sinon. Vous ne pouvez pas utiliser la fonction `reverse`, ni les boucles `for` et `while`. Un mot est un *palindrome* s'il peut être lu indifféremment de gauche à droite ou de droite à gauche. Par exemple, "bob" et "abba" sont des palindromes, mais pas "abb".
- c) Une fonction **récursive** `longueur_paire(mot)` qui renvoie `True` si la chaîne `mot` est de longueur paire, et `False` sinon. Vous ne pouvez pas utiliser la fonction `len`. On rappelle que :
- la chaîne `mot` est de longueur 0 si et seulement si `mot == ''` renvoie `True`
 - la chaîne `mot` est de longueur 1 si et seulement si `mot[1 :] == ''` renvoie `True`.